

国家精品课程/ 国家精品资源共享课程/ 国家级精品教材

国家级十一(二)五规划教材/ 教育部自动化专业教学指导委员会牵头规划系列教材

控制系统仿真与CAD

第十一章：分数阶控制基础

Chapter 11 Fundamentals in Fractional-order Control



Professor Dingyu Xue, xuedingyu@mail.neu.edu.cn
School of Information Science and Engineering,
Northeastern University, Shenyang, CHINA



分数阶微积分学概述

- 分数阶微积分是传统微积分学的拓展，在很多科学与工程领域都有应用范例
- 本章主要内容
 - 分数阶微积分的定义
 - 分数阶微积分计算
 - 分数阶传递函数模型
 - 线性分数阶系统建模与分析
 - 分数阶PID控制器
 - 分数阶非线性系统的仿真分析

国家精品课程/ 国家精品资源共享课程/ 国家级精品教材

国家级十一(二)五规划教材/ 教育部自动化专业教学指导委员会牵头规划系列教材

控制系统仿真与CAD

第十一章 分数阶控制基础

分数阶微积分概述

An Introduction to Fractional Calculus



主讲：薛定宇教授



分数阶微积分学概述

- 第3章的整数阶微积分问题 $d^n y/dx^n$
- 1695年法国数学家 L'Hôpital 问过微积分学创造者之一 Leibniz 的问题, $n = 1/2$?
- 分数阶微积分理论已经有300年的历史了
 - 早期主要侧重于理论研究
 - 近年来在很多领域都已经开始应用
 - 自动控制领域出现了分数阶控制理论等新的分支
 - 分数阶信号处理



为什么我们需要分数阶微积分学？

- 例: 正弦函数的导数是什么? 高阶导数是什么?

$$\frac{d^n}{dt^n} \sin t = \sin \left(t + n \frac{\pi}{2} \right)$$

- 分数阶导数提供的信息



```
>> n0=0:0.1:1.5; t=0:0.2:2*pi; Z=[];  
    for n=n0, Z=[Z; sin(t+n*pi/2)]; end  
    surf(t,n0,Z)
```

- 分数阶微积分可能揭示出整数阶微积分视角下看不到的东西



分数阶微积分的定义

- 四种常用的定义、相互关联
 - 分数阶Cauchy积分公式
 - Grünwald-Letnikov分数阶微积分定义
 - Riemann-Liouville分数阶微积分公式
 - Caputo分数阶微分定义



分数阶**Cauchy**积分公式

➤ 数学描述

$$\mathcal{D}^\gamma f(t) = \frac{\Gamma(\gamma + 1)}{2\pi j} \oint_C \frac{f(\tau)}{(\tau - t)^{\gamma+1}} d\tau$$

- 其中，C为包围 $f(t)$ 单值与解析开区域的光滑曲线
- 不易计算，不考虑



Grünwald–Letnikov分数阶微积分定义

➤ 数学描述

$${}_a\mathcal{D}_t^\alpha f(t) = \lim_{h \rightarrow 0} \frac{1}{h^\alpha} \sum_{j=0}^{[(t-a)/h]} (-1)^j \binom{\alpha}{j} f(t - jh)$$

➤ 其中， $\binom{\alpha}{j}$ 为二项式系数 $w_j^{(\alpha)} = (-1)^j \binom{\alpha}{j}$

$$w_0^{(\alpha)} = 1, \quad w_j^{(\alpha)} = \left(1 - \frac{\alpha + 1}{j}\right) w_{j-1}^{(\alpha)}, \quad j = 1, 2, \dots$$



Riemann–Liouville 分数阶微积分公式

➤ 积分的数学描述

$${}_a\mathcal{D}_t^{-\alpha} f(t) = \frac{1}{\Gamma(\alpha)} \int_a^t (t - \tau)^{\alpha-1} f(\tau) d\tau$$

➤ 其中 , $0 < \alpha < 1$, 且 a 为初始时间

➤ 分数阶微分定义的数学描述

$${}_a\mathcal{D}_t^{\beta} f(t) = \frac{d^n}{dt^n} \left[{}_a\mathcal{D}_t^{-(n-\beta)} f(t) \right] = \frac{1}{\Gamma(n - \beta)} \frac{d^n}{dt^n} \left[\int_a^t \frac{f(\tau)}{(t - \tau)^{\beta-n+1}} d\tau \right]$$



Caputo 分数阶微分定义

➤ 微分的数学描述：

$${}_a\mathcal{D}_t^\beta f(t) = \frac{1}{\Gamma(n-\beta)} \frac{d^n}{dt^n} \left[\int_a^t \frac{f(\tau)}{(t-\tau)^{\beta-n+1}} d\tau \right]$$

$${}_0\mathcal{D}_t^\alpha y(t) = \frac{1}{\Gamma(1-\alpha)} \int_0^t \frac{y^{(m+1)}(\tau)}{(t-\tau)^\alpha} d\tau$$

➤ 其中， $\alpha = m + \gamma$ ， m 为整数， $0 < \gamma \leq 1$

➤ 当 $\gamma < 0$ 时，Caputo 分数阶积分的数学描述

➤ 与 Riemann–Liouville 积分定义完全一致

$${}_0\mathcal{D}_t^\gamma y(t) = \frac{1}{\Gamma(-\gamma)} \int_0^t \frac{y(\tau)}{(t-\tau)^{1+\gamma}} d\tau$$



不同分数阶微积分定义的关系与性质

➤ Grünwald–Letnikov, Riemann–Liouville 等效

➤ Caputo 定义和 Riemann–Liouville 定义

$$\begin{aligned} \text{➤ } 0 < \alpha < 1 \quad {}^C_{t_0} \mathcal{D}_t^\alpha y(t) &= {}^{\text{RL}}_{t_0} \mathcal{D}_t^\alpha (y(t) - y(t_0)) \\ &= {}^{\text{RL}}_{t_0} \mathcal{D}_t^\alpha y(t) - \frac{y(t_0)(t - t_0)^{-\alpha}}{\Gamma(1 - \alpha)} \end{aligned}$$

➤ 更一般地

$${}^C_{t_0} \mathcal{D}_t^\alpha y(t) = {}^{\text{RL}}_{t_0} \mathcal{D}_t^\alpha y(t) - \sum_{k=0}^{m-1} \frac{y^{(k)}(t_0)}{\Gamma(k - \alpha + 1)} (t - t_0)^{k - \alpha}$$



分数阶微积分的性质

- 解析函数 $f(t)$, ${}_0\mathcal{D}_t^\alpha f(t)$ 对 t 和 α 都是解析的
- α 为整数时, 分数阶微分与整数阶微分的值完全一致, 且 ${}_0\mathcal{D}_t^0 f(t) = f(t)$

- 分数阶微积分算子为线性的, 对常数 a 、 b

$${}_0\mathcal{D}_t^\alpha [af(t) + bg(t)] = a {}_0\mathcal{D}_t^\alpha f(t) + b {}_0\mathcal{D}_t^\alpha g(t)$$

- 分数阶微积分算子满足交换律

$${}_0\mathcal{D}_t^\alpha \left[{}_0\mathcal{D}_t^\beta f(t) \right] = {}_0\mathcal{D}_t^\beta \left[{}_0\mathcal{D}_t^\alpha f(t) \right] = {}_0\mathcal{D}_t^{\alpha+\beta} f(t)$$



函数分数阶导数的**Laplace**变换

- 函数的分数阶积分表达式的 Laplace 变换

$$\mathcal{L}\left[\mathcal{D}_t^{-\gamma} f(t)\right] = s^{-\gamma} \mathcal{L}[f(t)]$$

- Riemann-Liouville 微分的 Laplace 变换为

$$\mathcal{L}\left[{}^{\text{RL}}_0 \mathcal{D}_t^{\alpha} f(t)\right] = s^{\alpha} \mathcal{L}[f(t)] - \sum_{k=1}^{n-1} s^k \left[{}_0 \mathcal{D}_t^{\alpha-k-1} f(t) \right]_{t=0}$$

- Caputo 微分的Laplace变换

$$\mathcal{L}\left[{}_0^{\text{C}} \mathcal{D}_t^{\gamma} f(t)\right] = s^{\gamma} F(s) - \sum_{k=0}^{n-1} s^{\gamma-k-1} f^{(k)}(0)$$



分数阶微积分定义小结

- 引入分数阶的视角可能看到整数阶微积分框架下看不到的东西
- 分数阶微积分的几种定义
 - 主要探讨两种定义 Riemann-Liouville、Caputo
 - Caputo定义的初值问题更接近实际



国家精品课程/ 国家精品资源共享课程/ 国家级精品教材

国家级十一(二)五规划教材/ 教育部自动化专业教学指导委员会牵头规划系列教材

控制系统仿真与CAD

第十一章 分数阶控制基础

分数阶微积分计算

Computations in Fractional Calculus



主讲：薛定宇教授



分数阶微积分的数值计算

- 原函数或其采样点已知时
 - 用 Grunwald–Letnikov 定义直接计算
 - Grunwald–Letnikov 定义下高精度数值求解
 - Caputo 定义的高精度数值解
- 原函数未知时
 - 通过高阶整数阶滤波器去模拟分数阶微积分行为
 - Oustaloup 滤波器逼近 Grunwald–Letnikov 微积分
 - Caputo 微积分的滤波器逼近



用Grünwald-Letnikov定义求解分数阶微分

➤ 数学描述

$$\begin{aligned} {}_a\mathcal{D}_t^\alpha f(t) &= \lim_{h \rightarrow 0} \frac{1}{h^\alpha} \sum_{j=0}^{[(t-a)/h]} (-1)^j \binom{\alpha}{j} f(t - jh) \\ &\approx \frac{1}{h^\alpha} \sum_{j=0}^{[(t-a)/h]} w_j^{(\alpha)} f(t - jh) \end{aligned}$$

➤ 其中

$$w_0^{(\alpha)} = 1, w_j^{(\alpha)} = \left(1 - \frac{\alpha + 1}{j}\right) w_{j-1}^{(\alpha)}, j = 1, 2, \dots$$



分数阶微分数值计算

➤ 数学描述

$${}_a\mathcal{D}_t^\alpha f(t) \approx \frac{1}{h^\alpha} \sum_{j=0}^{[(t-a)/h]} w_j^{(\alpha)} f(t-jh)$$
$$w_0^{(\alpha)} = 1, w_j^{(\alpha)} = \left(1 - \frac{\alpha+1}{j}\right) w_{j-1}^{(\alpha)}, j = 1, 2, \dots$$

➤ 构造一个MATLAB函数 $o(h)$

```
function dy=glfdiff(y,t,gam)
h=t(2)-t(1); dy(1)=0; y=y(:); t=t(:); w=1;
for j=2:length(t), w(j)=w(j-1)*(1-(gam+1)/(j-1)); end
for i=2:length(t), dy(i)=w(1:i)*[y(i:-1:1)]/h^gam; end
```

➤ 函数调用格式 $y_1 = \text{glfdiff}(y, t, \gamma)$



高精度分数阶微积分数值计算

➤ 高精度Grunwald–Letnikov微积分的数值计算

➤ 算法精度 $o(h^p)$

➤ MATLAB函数 $[y_1, t] = \text{glfdiff9}(y, t, \gamma, p)$

➤ Caputo微积分的高精度数值计算

$$y_1 = \text{caputo9}(y, t, \alpha, p)$$

➤ 代码下载——FOTF工具箱

➤ <http://cn.mathworks.com/matlabcentral/fileexchange/60874-fotf-toolbox>



例11-2 常数的微积分

- 阶跃函数的导数和积分是什么？
- 整数阶的积分与微分
- 对不同的阶次



```
>> t=0:0.01:1.5;  
    gam=[-1 -0.5 0.3 0.5 0.7];  
    y=ones(size(t)); dy=[];  
    for a=gam, dy=[dy; glfdiff(y,t,a)]; end  
    plot(t,dy)
```



Caputo微积分定义的数值计算的精度评价

➤ 计算指数函数 $\exp(-t)$ 的0.6阶Caputo导数 ($h=0.01$)

➤ 解析解 $y_0(t) = {}_0^C \mathcal{D}_t^\alpha e^{-t} = \lambda^q t^\gamma \mathcal{E}_{1,\gamma+1}(\lambda t) = -t^{0.4} \mathcal{E}_{1,1.4}(-t)$

➤ Mittag-Leffler函数

$$y = \text{ml_func}([\alpha, \beta], z) \quad \mathcal{E}_{\alpha, \beta}(z) = \sum_{k=0}^{\infty} \frac{z^k}{\Gamma(\alpha k + \beta)}$$

➤ MATLAB直接求解



```
>> t0=0.5:0.5:5; q=1; gam=q-0.6;  
y0=-t0.^0.4.*ml_func([1,1.4],-t0,0,eps);  
t=0:0.01:5; y=exp(-t); ii=[51:50:501]; T=t0';  
for p=1:6, y1=caputo9(y,t,0.6,p); T=[T [y1(ii)-y0']]; end
```



数值微分计算的精度演示

➤ 选择步长 $h=0.01$

time	$p = 1$	$p = 2$	$p = 3$	$p = 4$	$p = 5$	$p = 6$
0.5	-0.0018	1.19×10^{-5}	-8.89×10^{-8}	7.07×10^{-10}	-5.85×10^{-12}	4×10^{-14}
1	-0.00172	1.15×10^{-5}	-8.59×10^{-8}	6.85×10^{-10}	-5.69×10^{-12}	4×10^{-14}
1.5	-0.00151	1.01×10^{-5}	-7.52×10^{-8}	6.00×10^{-10}	-4.99×10^{-12}	4×10^{-14}
2	-0.00129	8.61×10^{-6}	-6.45×10^{-8}	5.15×10^{-10}	-4.28×10^{-12}	5×10^{-14}
2.5	-0.0011	7.39×10^{-6}	-5.53×10^{-8}	4.41×10^{-10}	-3.7×10^{-12}	1×10^{-14}
3	-0.00096	6.40×10^{-6}	-4.78×10^{-8}	3.82×10^{-10}	-3.25×10^{-12}	2×10^{-14}
3.5	-0.00084	5.61×10^{-6}	-4.20×10^{-8}	3.35×10^{-10}	-2.96×10^{-12}	1.4×10^{-13}
4	-0.00075	4.99×10^{-6}	-3.73×10^{-8}	2.98×10^{-10}	-2.92×10^{-12}	1×10^{-14}
4.5	-0.00068	4.50×10^{-6}	-3.37×10^{-8}	2.68×10^{-10}	-2.72×10^{-12}	4.7×10^{-13}
5	-0.00062	4.11×10^{-6}	-3.07×10^{-8}	2.45×10^{-10}	-3.41×10^{-12}	5.3×10^{-13}



大步长的计算

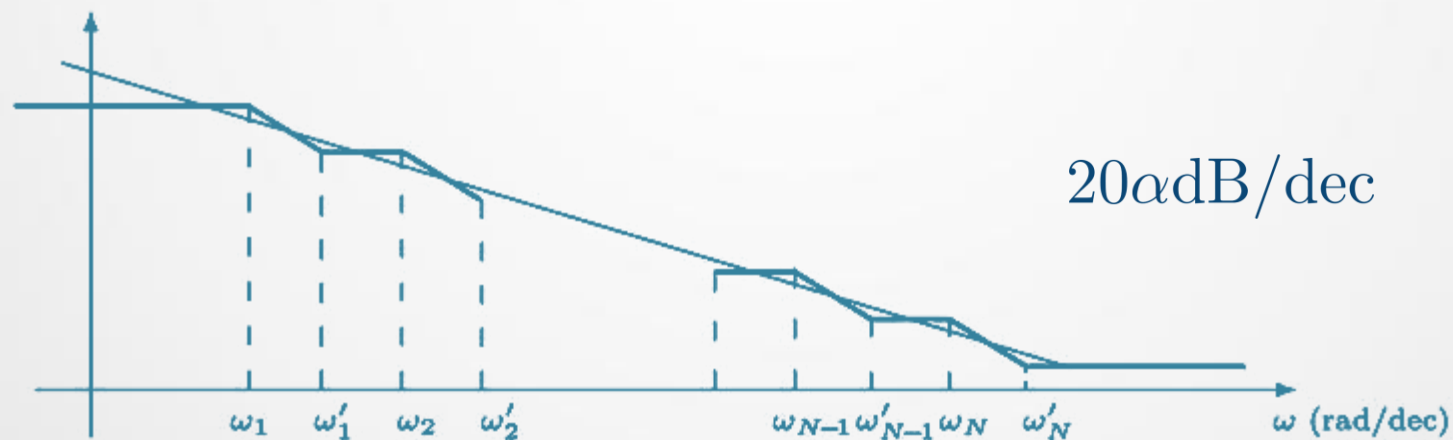
➤ 大步长 $h=0.1$

time	$p = 4$	$p = 5$	$p = 6$	$p = 7$	$p = 8$	$p = 9$
0.5	4.53×10^{-6}	-1.98×10^{-7}	-3.07×10^{-9}	-8.17×10^{-11}	-2.97×10^{-12}	-1.3×10^{-13}
1	5.98×10^{-6}	-4.73×10^{-7}	3.74×10^{-8}	-2.91×10^{-9}	2.48×10^{-10}	-2.03×10^{-11}
1.5	5.51×10^{-6}	-4.47×10^{-7}	3.73×10^{-8}	-3.12×10^{-9}	2.48×10^{-10}	-2.10×10^{-11}
2	4.77×10^{-6}	-3.90×10^{-7}	3.28×10^{-8}	-2.85×10^{-9}	2.53×10^{-10}	-1.63×10^{-11}
2.5	4.08×10^{-6}	-3.34×10^{-7}	2.81×10^{-8}	-2.45×10^{-9}	2.50×10^{-10}	-4.51×10^{-11}
3	3.51×10^{-6}	-2.87×10^{-7}	2.41×10^{-8}	-2.06×10^{-9}	1.52×10^{-10}	-6.34×10^{-12}
3.5	3.05×10^{-6}	-2.48×10^{-7}	2.08×10^{-8}	-1.76×10^{-9}	4.49×10^{-11}	2.35×10^{-10}
4	2.69×10^{-6}	-2.18×10^{-7}	1.82×10^{-8}	-1.55×10^{-9}	2.27×10^{-10}	-6.05×10^{-10}
4.5	2.40×10^{-6}	-1.95×10^{-7}	1.62×10^{-8}	-1.39×10^{-9}	4.94×10^{-10}	-1.91×10^{-9}
5	2.17×10^{-6}	-1.76×10^{-7}	1.46×10^{-8}	-1.26×10^{-9}	-2.17×10^{-10}	1.14×10^{-8}



事先未知函数的分数阶导数

- 可以考虑采用高阶整数阶滤波器逼近分数阶算子
- 分数阶微分环节的频域响应（Bode图）
- 可以考虑引入整数阶滤波器去逼近
 - 渐近线逼近，实际逼近效果会更好





Oustaloup滤波算法

- 如果函数 $f(t)$ 的表达式不是事先已知
- 连续滤波器传递函数模型：Oustaloup算法
 - 感兴趣区间 (ω_b, ω_h)

$$G_f(s) = K \prod_{k=1}^N \frac{s + \omega'_k}{s + \omega_k}$$

$$\omega'_k = \omega_b \omega_u^{(2k-1-\gamma)/N}, \quad \omega_k = \omega_b \omega_u^{(2k-1+\gamma)/N}, \quad K = \omega_h^\gamma,$$

$$\omega_u = \sqrt{\omega_h / \omega_b}$$



Oustaloup 滤波器设计

➤ 数学表示 $G_f(s) = K \prod_{k=1}^N \frac{s + \omega'_k}{s + \omega_k} \quad \omega_u = \sqrt{\omega_h/\omega_b}$

$$\omega'_k = \omega_b \omega_u^{(2k-1-\gamma)/N}, \quad \omega_k = \omega_b \omega_u^{(2k-1+\gamma)/N}, \quad K = \omega_h^\gamma,$$

➤ 构造MATLAB函数，设计连续滤波器

```
function G=ousta_fod(gam,N,wb,wh)
k=1:N; wu=sqrt(wh/wb);
wkp=wb*wu.^((2*k-1-gam)/N); wk=wb*wu.^((2*k-1+gam)/N);
G=zpk(-wkp,-wk,wh^gam); G=tf(G);
```

➤ 函数调用格式 $G = \text{ousta_fod}(\gamma, N, \omega_b, \omega_h)$



例11-4 由滤波器计算微分

- 函数 $f(t) = e^{-t} \sin(3t + 1)$
- 感兴趣频率区间 $\omega_b = 0.01, \omega_h = 1000 \text{ rad/sec}$
- 分数阶阶次 0.5 , 5阶滤波器
- MATLAB求解语句：



```
>> G=ousta_fod(0.5,5,0.01,1000), bode(G)
```

- 绘制分数阶微分曲线：



```
>> t=0:0.001:pi; y=exp(-t).*sin(3*t+1);  
    y1=lsim(G,y,t); y2=glfdiff(y,t,0.5); plot(t,y1,t,y2)
```



非零初值Caputo微积分的Oustaloup 滤波器逼近

- 标准Oustaloup滤波器可以用于RL微积分逼近
- 直接方法构造Caputo微分器较麻烦，不宜采用
- 非零初值信号的Caputo微积分逼近可以根据两个定理
 - 定理1 RL积分器

$${}_t^C \mathcal{D}_t^\gamma y(t) = {}_t^{\text{RL}} \mathcal{D}_t^{-(\lceil \gamma \rceil - \gamma)} \left[y^{(\lceil \gamma \rceil)}(t) \right] \quad {}_t^C \mathcal{D}_t^{2.3} y(t) = {}_t^{\text{RL}} \mathcal{D}_t^{-0.7} [y'''(t)]$$

- 定理2 RL微分器

$${}_t^{\text{RL}} \mathcal{D}_t^{\lceil \gamma \rceil - \gamma} [{}_t^C \mathcal{D}_t^\gamma y(t)] = y^{(\lceil \gamma \rceil)}(t) \quad {}_t^{\text{RL}} \mathcal{D}_t^{0.7} [{}_t^C \mathcal{D}_t^{2.3} y(t)] = y'''(t)$$



分数阶微积分计算小结

- 如果信号的采样点已知
 - Riemann-Liouville glfdiff, glfdiff9
 - Caputo微积分 caputo9
- 如果信号未知
 - Oustaloup滤波器
 - Caputo微积分的Oustaloup滤波器实现



国家精品课程/ 国家精品资源共享课程/ 国家级精品教材

国家级十一(二)五规划教材/ 教育部自动化专业教学指导委员会牵头规划系列教材

控制系统仿真与CAD

第十一章 分数阶控制基础

分数阶传递函数模型与分析(上)

Modeling and Analysis of Fractional-order Transfer Functions (II)



主讲：薛定宇教授



分数阶传递函数模型

➤ 数学形式

$$G(s) = \frac{b_1 s^{\gamma_1} + b_2 s^{\gamma_2} + \cdots + b_m s^{\gamma_m}}{a_1 s^{\eta_1} + a_2 s^{\eta_2} + \cdots + a_{n-1} s^{\eta_{n-1}} + a_n s^{\eta_n}} e^{-T s}$$

➤ 多变量系统

$$G(s) = \begin{bmatrix} g_{11}(s) & \cdots & g_{1p}(s) \\ \vdots & \ddots & \vdots \\ g_{q1}(s) & \cdots & g_{qp}(s) \end{bmatrix}$$

➤ 需要一个分数阶传递函数的“类”



面向对象的程序设计举例

——MATLAB下类的设计

- 需要建一个文件夹 @fotf
 - 设计域（成员变量）——num, nn, den, nd, ioDelay
 - 与其相关的所有函数全部置于此文件夹内
 - 必要的文件
 - 类的创建——fotf.m
 - 类的显示——display.m
 - 其他重载函数，尽量与MATLAB控制系统工具箱的函数保持同名，尽量保持同样（相似）的调用格式，使得熟悉整数阶系统的使用者可以不费气力地直接使用分数阶系统工具箱
 - 注意别置入其他文件夹，因为可能会影响其他类的使用



例：类创建文件的内容

➤ 程序结构

```
classdef fotf
    properties
        num, nn, den, nd, ioDelay
    end
    methods
        function G=fotf(a,na,b,nb,T)
            if isa(a,'fotf'), G=a;
            elseif isa(a,'foss'), G=foss2fotf(a);
            elseif nargin==1 & (isa(a,'tf')|isa(a,'ss')|isa(a,'double')),
                a=tf(a); [n1,m1]=size(a); G=[]; D=a.ioDelay;
                for i=1:n1, g=[]; for j=1:m1,
                    [n,d]=tfdata(tf(a(i,j)),'v'); nn=length(n)-1:-1:0;
                    nd=length(d)-1:-1:0; g=[g fotf(d,nd,n,nn,D(i,j))];
                end, G=[G; g]; end
        end
    end
end
```



➤ 代码(续)

```
elseif nargin==1 & a=='s', G=fotf(1,0,1,1,0);  
else, ii=find(abs(a)<eps); a(ii)=[]; na(ii)=[];  
      ii=find(abs(b)<eps); b(ii)=[]; nb(ii)=[];  
      if length(b)==0, b=0; nb=0; end  
      if nargin==4, T=0; end  
      G.num=b; G.den=a; G.nn=nb; G.nd=na; G.ioDelay=T;  
end, end, end, end
```

➤ 函数调用方法

➤ 分数阶传递函数输入 $G = \text{fotf}(a, n_a, b, n_b, T)$

➤ s算子输入 $s = \text{fotf}('s')$

➤ 模型的转换 $G = \text{fotf}(G_1)$



例11-5 分数阶传递函数的输入

➤ 分数阶传递函数模型

$$G(s) = \frac{0.8s^{1.2} + 2}{1.1s^{1.8} + 1.9s^{0.5} + 0.4} e^{-0.5s}$$

➤ 模型输入



```
>> G=fotf([1.1,1.9,0.4],[1.8,0.5,0],...  
          [0.8,2],[1.2,0],0.5)
```



例11-6 分数阶传递函数矩阵的输入

➤ 数学模型

$$G(s) = \begin{bmatrix} \frac{1}{1.5s^{1.2} + 0.7} e^{-0.5s} & \frac{2}{1.2s^{1.1} + 1} e^{-0.2s} \\ \frac{3}{0.7s^{1.3} + 1.5} & \frac{2}{1.3s^{1.1} + 0.6} e^{-0.2s} \end{bmatrix}$$

➤ 输入语句



```
>> g1=fotf([1.5 0.7],[1.2 0],1,0,0.5);  
g2=fotf([1.2 1],[1.1 0],2,0,0.2);  
g3=fotf([0.7 1.5],[1.3 0],3,0);  
g4=fotf([1.3 0.6],[1.1 0],2,0,0.2);  
G=[g1,g2; g3,g4]
```



例11-7 重载函数编写举例

➤ 串联连接——置于@fotf

$$G(s) = G_1(s)G_2(s) = \frac{N_1(s)N_2(s)}{D_1(s)D_2(s)}$$

➤ MIMO算法 (矩阵乘法)

```
function G=mtimes(G1,G2)
G1=fotf(G1); G2=fotf(G2); [n1,m1]=size(G1); [n2,m2]=size(G2);
if m1==n2, G=fotf(zeros(n1,m2));
    for i=1:n1, for j=1:m2,
        for k=1:m1, G(i,j)=G(i,j)+sisofotftimes(G1(i,k),G2(k,j));
        end, end, end
elseif n1*m1==1, G=fotf(zeros(n2,m2));
    for i=1:n2, for j=1:m2, G(i,j)=G1*G2(i,j); end, end
elseif n2*m2==1, G=fotf(zeros(n1,m1));
    for i=1:n1, for j=1:m1, G(i,j)=G2*G1(i,j); end, end
else,
    error('The two matrices are incompatible for multiplication')
end
```



例11-8 多变量系统的闭环模型

➤ 开环模型

$$G(s) = \begin{bmatrix} \frac{1}{1.5s^{1.2} + 0.7} & \frac{2}{1.2s^{1.1} + 1} \\ \frac{3}{0.7s^{1.3} + 1.5} & \frac{2}{1.3s^{1.1} + 0.6} \end{bmatrix}$$

➤ 闭环模型输入



```
>> g1=fotf([1.5 0.7],[1.2 0],1,0);  
g2=fotf([1.2 1],[1.1 0],2,0);  
g3=fotf([0.7 1.5],[1.3 0],3,0);  
g4=fotf([1.3 0.6],[1.1 0],2,0);  
G=[g1,g2; g3,g4]; H=fotf(eye(2)); G1=feedback(G,H)
```



分数阶线性系统的分析

- 如果已知系统的模型（ FOTF 或 FOSS 对象 ） G
 - 性质分析
 - `isstable(G)`, `norm(G)`, `norm(G,inf)`
 - 时域分析
 - `step(G,t)` , `impulse(G,t)` , `lsim(G,u,t)`
 - 频域分析
 - `bode(G,w)` , `nyquist(G,w)` , `michols(G,w)` , `margin`
 - 多变量频域响应分析： `gershgorin(G)` , `mfrd(G,w)` , `sigma(G,w)`
 - 根轨迹分析 `rlocus(G)`



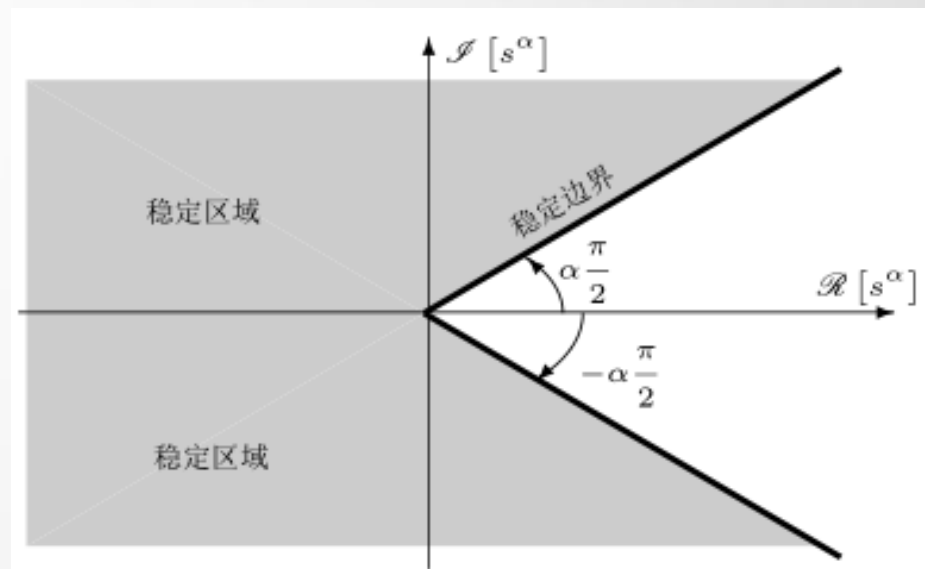
分数阶系统的稳定性判定

- 成比例阶系统的基阶为 α 。记 $\lambda = s^\alpha$ ，则原分数阶传递函数 $G(s)$ 可以转换成关于 λ 的整数阶传递函数

$$G(\lambda) = \frac{b_1 \lambda^m + b_2 \lambda^{m-1} + \cdots + b_m \lambda + b_{m+1}}{a_1 \lambda^n + a_2 \lambda^{n-1} + \cdots + a_n \lambda + a_{n+1}}$$

- 系统稳定的判据 $|\arg(p_i)| > \alpha \frac{\pi}{2}$

- 稳定边界对 λ 而言是斜线;
- 对 s 而言仍然是虚轴





例11-9 分数阶系统的分析

➤ 系统模型
$$G(s) = \frac{-2s^{0.63} - 4}{2s^{3.501} + 3.8s^{2.42} + 2.6s^{1.798} + 2.5s^{1.31} + 1.5}$$

➤ 系统的分析方法

➤ 稳定性分析



```
>> a=[2,3.8,2.6,2.5,1.5]; na=[3.501,2.42,1.798,1.31,0];  
    b=[-2,-4]; nb=[0.63,0]; G=fotf(a,na,b,nb);  
    [key,alpha,a1,p]=isstable(G), p1=p.^(1/alpha)
```

➤ 范数计算



```
>> n1=norm(G), n2=norm(G,inf)
```



分数阶系统分析 (续)

➤ 频域响应分析



```
>> w=logspace(-2,2); bode(G,w);
```



```
>> w=logspace(-2,4,400); nyquist(G,w); grid  
[Gm,Pm,Wcg,Wcp]=margin(G)
```

➤ 时域响应



```
>> t=0:0.01:30; step(G,t); y=impulse(G,t); line(t,y);
```

➤ 近似根轨迹分析

➤ 不可能绘制精确根轨迹(3501阶系统), 只能绘制近似根轨迹



```
>> b=[-2 -4]; nb=[0.6 0]; a=[2 3.8 2.6 2.5 1.5];  
na=[3.5 2.4 1.8 1.3 0]; G1=fotf(a,na,b,nb); rlocus(G1)
```



多变量系统的频域分析

- 多变量系统设计的难处
 - 不同输入输出对之间的耦合
 - 应该考虑解耦算法，或将其补偿为对角占优的系统，再设计控制器，后面将介绍
- 对角占优 $\text{gershgorin}(H)$
 - Gershgorin定理
 - 对角占优判定：绘制Nyquist图加Gershgorin带，如果Gershgorin带不包围坐标原点，则系统为对角占优系统
 - 如果对角占优，则可以考虑单回路控制器设计



例11-10 多变量系统的对角占优分析

➤ 多变量系统

$$G(s) = \begin{bmatrix} \frac{1}{1.35s^{1.2} + 2.3s^{0.9} + 1} & \frac{2}{4.13s^{0.7} + 1} \\ \frac{1}{0.52s^{1.5} + 2.03s^{0.7} + 1} & -\frac{1}{3.8s^{0.8} + 1} \end{bmatrix}$$

➤ 补偿模型

$$K_p = \begin{bmatrix} 1/3 & 2/3 \\ 1/3 & -1/3 \end{bmatrix}, \quad K_d(s) = \begin{bmatrix} 1/(2.5s + 1) & 0 \\ 0 & 1/(3.5s + 1) \end{bmatrix}$$

➤ 对角占优分析



```
>> s=fotf('s'); g1=1/(1.35*s^1.2+2.3*s^0.9+1);  
g2=2/(4.13*s^0.7+1); g3=1/(0.52*s^1.5+2.03*s^0.7+1);  
g4=-1/(3.8*s^0.8+1); G0=[g1,g2; g3,g4];  
w=logspace(-1,0); H=mfrd(G0,w); gershgorin(H)
```



```
>> Kp=[1/3 2/3; 1/3 -1/3]; s=tf('s');  
Kd=[1/(2.5*s+1), 0; 0, 1/(3.5*s+1)];  
K1=G0*fotf(Kd)*fotf(Kp); H2=mfrd(K1,w); gershgorin(H2)
```



分数阶系统分析小结

- 仿照整数阶控制系统工具箱
 - 类的设计——名字、域名
 - 重载函数的设计——尽量与整数阶系统一致
- 分数阶传递函数类的使用
 - 一批重载函数直接可用—— $*$, $+$, feedback ,
isstable, bode ,Nyquist, nichols, rlocus
step, impule, gershgorin



国家精品课程/ 国家精品资源共享课程/ 国家级精品教材

国家级十一(二)五规划教材/ 教育部自动化专业教学指导委员会牵头规划系列教材

控制系统仿真与CAD

第十一章 分数阶控制基础

分数阶PID控制器

Fractional-order PID Controllers



主讲：薛定宇教授



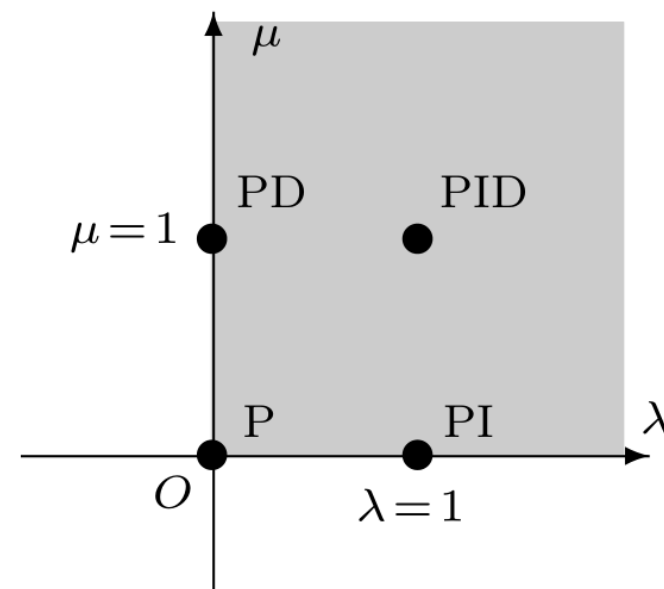
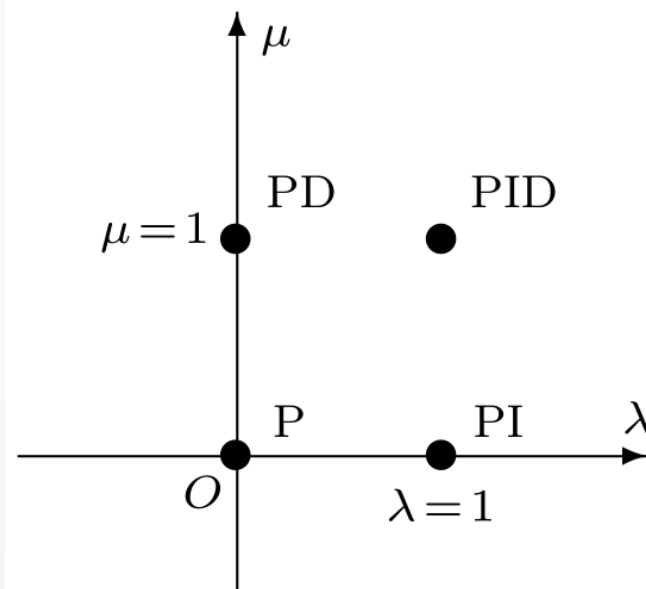
分数阶**PID**控制器设计

- 分数阶PID控制器的结构
- 分数阶PID控制器的设计方法
 - 基于频域响应的设计方法
 - 基于数值最优化的设计方法
 - 最优分数阶PID控制器设计界面



分数阶**PID**控制器

- 数学形式 $G_c(s) = K_p + \frac{K_i}{s^\lambda} + K_d s^\mu$
- Igor Podlubny教授





分数阶**PID**控制器输入

➤ 编写函数 $G_c(s) = K_p + \frac{K_i}{s^\lambda} + K_d s^\mu$

```
function Gc=fopid(Kp,Ki,Kd,lam,mu0)
s=fotf('s');
if length(Kp)==1,
    Gc=Kp+Ki*s^(-lam)+Kd*s^mu0;
else,
    x=Kp, Gc=x(1)+x(2)*s^(-x(4))+x(3)*s^(x(5));
end
```

➤ 调用格式 $G_c = \text{fopid}(K_p, K_i, K_d, \lambda, \mu)$



例11-10 最优分数阶PID控制器设计

➤ 受控对象 $G(s) = \frac{1}{s^{2.6} + 2.2s^{1.5} + 2.9s^{1.3} + 3.32s^{0.9} + 1}$

➤ 目标函数

```
function fy=fpidfun(x,G,t,key)
C=fopid(x); dt=t(2)-t(1); e=step(feedback(1,G*C),t);
if key==1, fy=dt*sum(t.*abs(e)); else, fy=dt*sum(e.^2); end
disp([x(:).', fy])
```

➤ 直接寻优设计与仿真



```
>> s=fotf('s'); G=1/(s^2.6+3.3*s^1.5+2.9*s^1.3+3.32*s^0.9+1);
xm=zeros(5,1); xM=[30; 30; 30; 2; 2]; x0=rand(5,1)';
t=0:0.01:8; x=fminsearchbnd(@fpidfun,x0,xm,xM,[],G,t,1)
Gc1=fopid(x); step(feedback(G*Gc1,1),t);
```



通用的设计框架

➤ 目标函数

```
function [fy,C]=fpidfun(x)
global G t key1 key2; s=fotf('s'); t=t(:);
switch key1
    case {'fpid',1}, C=x(1)+x(2)*s^(-x(4))+x(3)*s^(x(5));
    case {'fpi',2}, C=x(1)+x(2)*s^(-x(3));
    case {'fpd',3}, C=x(1)+x(2)*s^x(3);
    case {'fpidx',4}, C=x(1)+x(2)/s+x(3)*s^x(4);
    case {'pid','PID',5}, C=x(1)+x(2)/s+x(3)*s;
end
dt=t(2)-t(1); e=step(feedback(1,G*C),t);
switch key2
    case {'itae','ITAE',1}, fy=dt*abs(e)*t;
    case {'ise','ISE',2}, fy=dt*sum(e.^2);
    case {'iae','IAE',3}, fy=dt*sum(abs(e));
    case {'itse','ITSE',4}, fy=dt*e.^2*t;
    otherwise,
        error(['Available criteria are itae, ise, iae, itse.'])
end, disp([x(:).' fy])
```

➤ 求解函数 $[G_c, x, y]=\text{fpid}\text{tune}(x_0, x_m, x_M, k_A)$



最优分数阶**PID**控制器设计程序

- 需要给全局变量赋值
 - 受控对象 —— FOTF数据结构 G
 - 时间向量 t
 - 开关变量 key1 —— ‘fpi’, ‘fpid’, ‘fpd’, ‘fpidx’, ‘pid’
 - 开关key2 —— ‘itae’, ‘ise’, ‘iae’, ‘itse’
- 直接设计 $[G_c, x, y] = \text{fpidtune}(x_0, x_m, x_M, kA)$
 - kA —— 寻优算法, 区间 $\lambda, \mu \in (0, 2)$



例11-12 最优分数阶PID控制器

➤ 受控对象

$$G(s) = \frac{1}{s^{2.6} + 2.2s^{1.5} + 2.9s^{1.3} + 3.32s^{0.9} + 1}$$

➤ 最优设计



```
>> global G t key1 key2; s=fotf('s');  
G=1/(s^2.6+3.3*s^1.5+2.9*s^1.3+3.32*s^0.9+1);  
xm=zeros(5,1); xM=[30; 30; 30; 2; 2];  
x0=rand(5,1); t=0:0.01:8;  
key1='fpid'; key2='itae'; [Gc,x]=fpidtune(x0,xm,xM,1)
```



```
>> [Gc,x]=fpidtune(x,xm,xM,1)
```



与整数阶最优控制器比较

➤ 设计并比较整数阶PID控制器



```
>> xm=zeros(3,1); xM=[30; 30; 30]; x0=[1;1;1].';  
key1='pid'; key2='itae';  
[Gc1,x]=fpidtune(x0,xm,xM,1)  
step(feedback(G*Gc,1),t); hold on;  
step(feedback(G*Gc1,1),t);
```

➤ 遗留的问题：含有延迟的受控对象怎么办？

➤ FOTF不能处理闭环延迟模型

➤ 数值Laplace反变换 —— 速度较慢



例11-13 延迟系统的最优分数阶PID设计

➤ 目标函数描述

➤ 采用数值Laplace反变换为核心计算仿真

```
function fy=fun_opts(x)
global G t key1 key2; t0=t(1); t1=t(end); N=length(t);
dt=t(2)-t(1); Gf=get_fpidf(x,G,key1); U='1/s';
[t,y]=INVLAP_new(Gf,t0,t1,N,1,U); e=1-y;
switch key2
    case {'itae','ITAE',1}, fy=dt*sum(t.*abs(e));
    case {'ise','ISE',2}, fy=dt*sum(e.^2);
    case {'iae','IAE',3}, fy=dt*sum(abs(e));
end, disp([x(:).' fy])
```



控制器设计与仿真

➤ ITAE性能指标



```
>> clear; global G t key1 key2
G='exp(-s)/(0.8*s^2.2+0.5*s^0.9+1)'; key2='itae';
key1='fpid'; t=0.01:0.01:20; x0=rand(5,1);
xm=zeros(5,1); xM=[20;20;20;2;2];
x=fminsearchbnd(@fun_opts,x0,xm,xM)

>> t0=0.01; tn=20; N=2000; Gf=get_fpidf(x,G,'fpid');
[t1,y1]=INVLAP_new(Gf,t0,tn,N,1,'1/s'); plot(t1,y1)
```

➤ 直接设计



```
>> key2='ise'; x=fminsearchbnd(@fun_opts,x0,xm,xM);
U='1/s'; Gf=get_fpidf(x,G,'fpid');
[t2,y2]=INVLAP_new(Gf,t0,tn,N,1,U);
```



闭环系统效果比较

➤ 常规PID控制器设计

```
>> key1='pid'; key2='itae';  
x0=[1 1 1]'; xm=[0 0 0]'; xM=[30,30,30]';  
x=fminsearchbnd(@fun_opts,x0,xm,xM);  
Gf=get_fpidf(x,G,'pid');  
[t3,y3]=INVLAP_new(Gf,t0,tn,N,1,U);
```

➤ 仿真比较

```
>> plot(t1,y1,t2,y2,t3,y3)
```



例11-14 基于界面的设计

➤ 受控对象模型

$$G(s) = \frac{1}{0.8s^{2.2} + 0.5s^{0.9} + 1}$$

➤ 输入模型

 `>> G=fotf([0.8 0.5 1],[2.2 0.9 0],1,0)`

➤ 设计控制器

➤ 参数上界为 15 , 时间 (0,8)

 `>> optimfopid`



最优分数阶**PID**控制器小结

- 基于数值最优化的最优PID控制器设计
- 通过三个例子演示控制器设计
 - 一个简单的例子
 - 通用的无延迟受控对象的最优分数阶PID控制器设计
函数 `fpid tune`
 - 有延迟受控对象，通过数值Laplace反变换求解
- 最优分数阶PID控制器设计界面 `optimfopid`



控制系统仿真与 CAD

国家精品课程/ 国家精品资源共享课程/ 国家级精品教材

国家级十一(二)五规划教材/ 教育部自动化专业教学指导委员会牵头规划系列教材

控制系统仿真与CAD

第十一章 分数阶控制基础

分数阶非线性系统仿真 (上)

Simulation of Fractional-order Nonlinear Systems (I)



主讲：薛定宇教授



分数阶微分方程的求解

- 基于命令的求解方法
 - 零初值分数阶线性微分方程的闭式解法
 - 非零初值Caputo线性微分方程的数值求解
 - 非零初值Caputo线性微分方程的高精度求解
 - 非线性微分方程的高精度求解方法
- 基于框图的分数阶微分方程数值解



零初值分数阶线性微分方程的标准型

➤ 分数阶线性微分方程标准型

$$a_n \mathcal{D}_t^{\beta_n} y(t) + a_{n-1} \mathcal{D}_t^{\beta_{n-1}} y(t) + \cdots \\ + a_1 \mathcal{D}_t^{\beta_1} y(t) + a_0 \mathcal{D}_t^{\beta_0} y(t) = \hat{u}(t)$$

➤ 如果右侧不是 $u(t)$, 可以通过下式计算

$$\hat{u}(t) = b_1 \mathcal{D}_t^{\gamma_1} u(t) + b_2 \mathcal{D}_t^{\gamma_2} u(t) + \cdots + b_m \mathcal{D}_t^{\gamma_m} u(t)$$

➤ 假设 $\beta_n > \beta_{n-1} > \cdots > \beta_1 > \beta_0 > 0$

➤ 如果有负阶次 , 应该事先变换 $z(t) = \mathcal{D}_t^{\beta_0} y(t)$



线性微分方程的闭式解

➤ 由 Grünwald–Letnikov 定义

$${}_a\mathcal{D}_t^{\beta_i} y(t) \approx \frac{1}{h^{\beta_i}} \sum_{j=0}^{[(t-a)/h]} w_j^{(\beta_i)} y_{t-jh} = \frac{1}{h^{\beta_i}} \left[y_t + \sum_{j=1}^{[(t-a)/h]} w_j^{(\beta_i)} y_{t-jh} \right]$$

$$w_0^{(\beta_i)} = 1, \quad w_j^{(\beta_i)} = \left(1 - \frac{\beta_i + 1}{j} \right) w_{j-1}^{(\beta_i)}, \quad j = 1, 2, \dots$$

➤ 微分方程的闭式数值解为

$$y_t = \frac{1}{\sum_{i=0}^n \frac{a_i}{h^{\beta_i}}} \left[\hat{u}_t - \sum_{i=0}^n \frac{a_i}{h^{\beta_i}} \sum_{j=1}^{[(t-a)/h]} w_j^{(\beta_i)} y_{t-jh} \right]$$



分数阶微分方程求解函数

➤ 直接求解 $y = \text{fode_sol}(a, n_a, b, n_b, u, t)$

```
function y=fode_sol(a,na,b,nb,u,t)
h=t(2)-t(1); D=sum(a./[h.^na]);
nT=length(t); vec=[na nb]; W=[];
D1=b(:)./h.^nb(:); nA=length(a);
y1=zeros(nT,1); W=ones(nT,length(vec));
for j=2:nT, W(j,:)=W(j-1,:).*(1-(vec+1)/(j-1)); end
for i=2:nT,
    A=[y1(i-1:-1:1)]'*W(2:i,1:nA);
    y1(i)=(u(i)-sum(A.*a./[h.^na]))/D;
end
for i=2:nT, y(i)=(W(1:i,nA+1:end)*D1)'*[y1(i:-1:1)]; end
```



例11-17 分数阶微分方程的数值解

- 求解零初值分数阶线性微分方程

$$\mathcal{D}_t^{3.5}y(t) + 8\mathcal{D}_t^{3.1}y(t) + 26\mathcal{D}_t^{2.3}y(t) + 73\mathcal{D}_t^{1.2}y(t) + 90\mathcal{D}_t^{0.5}y(t) = 90 \sin t^2$$

- 选择计算步长 $h = 0.002$
- 求解，选择不同的 h 检验



```
>> a=[1,8,26,73,90]; n=[3.5,3.1,2.3,1.2,0.5];  
t=0:0.002:10; u=90*sin(t.^2);  
y=fode_sol(a,n,1,0,u,t); subplot(211),  
plot(t,y); subplot(212), plot(t,u)
```



分数阶微分方程的高精度求解

➤ Riemann–Liouville 方程的高精度求解

$$y = \text{fode_sol9}(a, n_a, b, n_b, u, t, p)$$

➤ Caputo方程的高精度求解

$$y = \text{fode_caputo9}(a, n_a, b, n_b, y_0, u, t, p)$$

➤ 非线性Caputo方程的高精度求解

➤ 预估解 $[y, t] = \text{nlfep}(\text{fun}, \alpha, y_0, t_n, h, p, \epsilon)$

➤ 校正解 $y = \text{nlfec}(\text{fun}, \alpha, y_0, y_p, t, p, \epsilon)$



例11-18 Caputo分数阶线性微分方程

➤ 已知线性微分方程

$$y'''(t) + \frac{1}{16} {}^C_0 \mathcal{D}_t^{2.5} y(t) + \frac{4}{5} y''(t) + \frac{3}{2} y'(t) + \frac{1}{25} {}^C_0 \mathcal{D}_t^{0.5} y(t) + \frac{6}{5} y(t) = \frac{172}{125} \cos \frac{4t}{5}$$

➤ 初值 $y(0) = 1, y'(0) = 4/5, y''(0) = -16/25, 0 \leq t \leq 30$

➤ 解析解 $y(t) = \sqrt{2} \sin(4t/5 + \pi/4)$

➤ MATLAB数值求解



```
>> a=[1 1/16 4/5 3/2 1/25 6/5]; na=[3 2.5 2 1 0.5 0];  
b=1; nb=0; t=[0:0.05:30]; u=172/125*cos(4*t/5);  
y0=[1 4/5 -16/25]; y1=fode_caputo9(a,na,b,nb,y0,u,t,4);  
y=sqrt(2)*sin(4*t/5+pi/4); max(abs(y-y1)), plot(t,y,t,y1)
```



例11-19 非线性微分方程的高精度求解

➤ Caputo定义下的微分方程

$${}_0^C \mathcal{D}_t^{1.455} y(t) = -t^{0.1} \frac{\mathcal{E}_{1,1.545}(-t)}{\mathcal{E}_{1,1.445}(-t)} e^t y(t) {}_0^C \mathcal{D}_t^{0.555} y(t) + e^{-2t} - [y'(t)]^2$$

➤初值 $y(0) = 1, y'(0) = -1$, 解析解 $y(t) = e^{-t}$



```
>> f=@(t,y,Dy)-t.^0.1.*ml_func([1,1.545],-t).*exp(t)./...  
    ml_func([1,1.445],-t).*y.*Dy(:,1)+exp(-2*t)-Dy(:,2).^2;  
alpha=[1.455,0.555,1]; y0=[1,-1]; tn=1; h=0.01; err=1e-8;  
[yp1,t]=nlfep(f,alpha,y0,tn,h,1,err);
```



```
>> tic, [y2,t]=nlfec(f,alpha,y0,yp1,t,2,err); toc  
max(abs(y2-exp(-t)))
```



国家精品课程/ 国家精品资源共享课程/ 国家级精品教材

国家级十一(二)五规划教材/ 教育部自动化专业教学指导委员会牵头规划系列教材

控制系统仿真与CAD

第十一章 分数阶控制基础

分数阶非线性系统仿真(下)

Simulation of Fractional-order Nonlinear Systems (II)



主讲：薛定宇教授



分数阶微分方程的求解

- 基于框图的分数阶微分方程数值解
 - 为什么需要基于框图的求解方法？
 - FOTF的分数阶微积分模块库
 - 通过例子演示框图解法



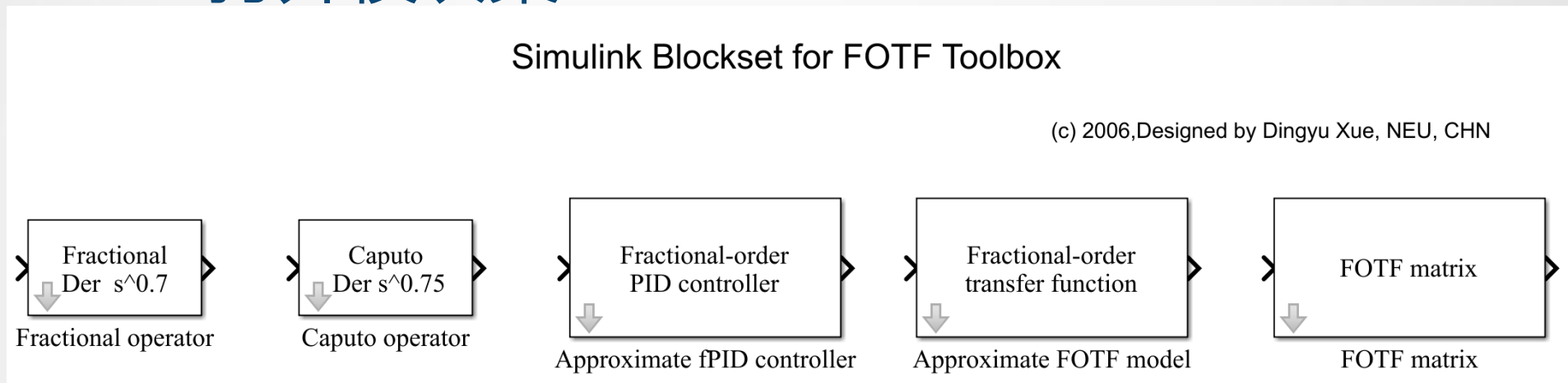
如何求解？ 如何评价？

- 复杂系统不宜用类似整数阶系统Runge-Kutta方法求解
 - 求解需要将整个微分方程组变成标准形式
 - 有时是不可行的，即使可行难度也极大
- 基于框图的解法应该是一个合理的选择
 - 很多方法只能求解零初值问题，非零初值如何求解？
- 如何评价？—— 基准测试问题
 - Xue Dingyu and Bai Lu. Benchmark problems for Caputo fractional-order ordinary differential equations. Fractional Calculus & Applied Analysis, 20(5), 2017



Simulink FOTF模块集

➤ 命令 fotflib 打开模块集



➤ 不建议使用的模块

➤ Caputo operator —— 应该使用性质加Oustaloup filter

➤ FOTF Matrix —— Approximate FOTF Model 更稳定



基准测试问题3 非线性非零初值的分数阶微分方程

- (基准问题3) 非线性非零初值的分数阶微分方程

$${}_0^C \mathcal{D}_t^{1.455} y(t) = -t^{0.1} \frac{\mathcal{E}_{1,1.545}(-t)}{\mathcal{E}_{1,1.445}(-t)} e^t y(t) {}_0^C \mathcal{D}_t^{0.555} y(t) + e^{-2t} - [y'(t)]^2$$

- 初始条件为 $y(0) = 1, y'(0) = -1$
- 时间区间 $0 \leq t \leq 1$
- 解析解 $y(t) = e^{-t}$
- 说明：该方程来源是Diethelm教授的著作，原书方程有误，解析解不满足微分方程



非线性分数阶微分方程求解

➤ 第3基准测试问题

$${}_0^C \mathcal{D}_t^{1.455} y(t) = -t^{0.1} \frac{\mathcal{E}_{1,1.545}(-t)}{\mathcal{E}_{1,1.445}(-t)} e^t y(t) {}_0^C \mathcal{D}_t^{0.555} y(t) + e^{-2t} - [y'(t)]^2$$

$$y(0) = 1, y'(0) = -1, 0 \leq t \leq 1, y(t) = e^{-t}$$

➤ 两个积分器串联即可

➤ 定义关键信号 $y(t), y'(t), y''(t)$

➤ 构造分数阶微分信号

➤ 闭环仿真求解



Simulink建模

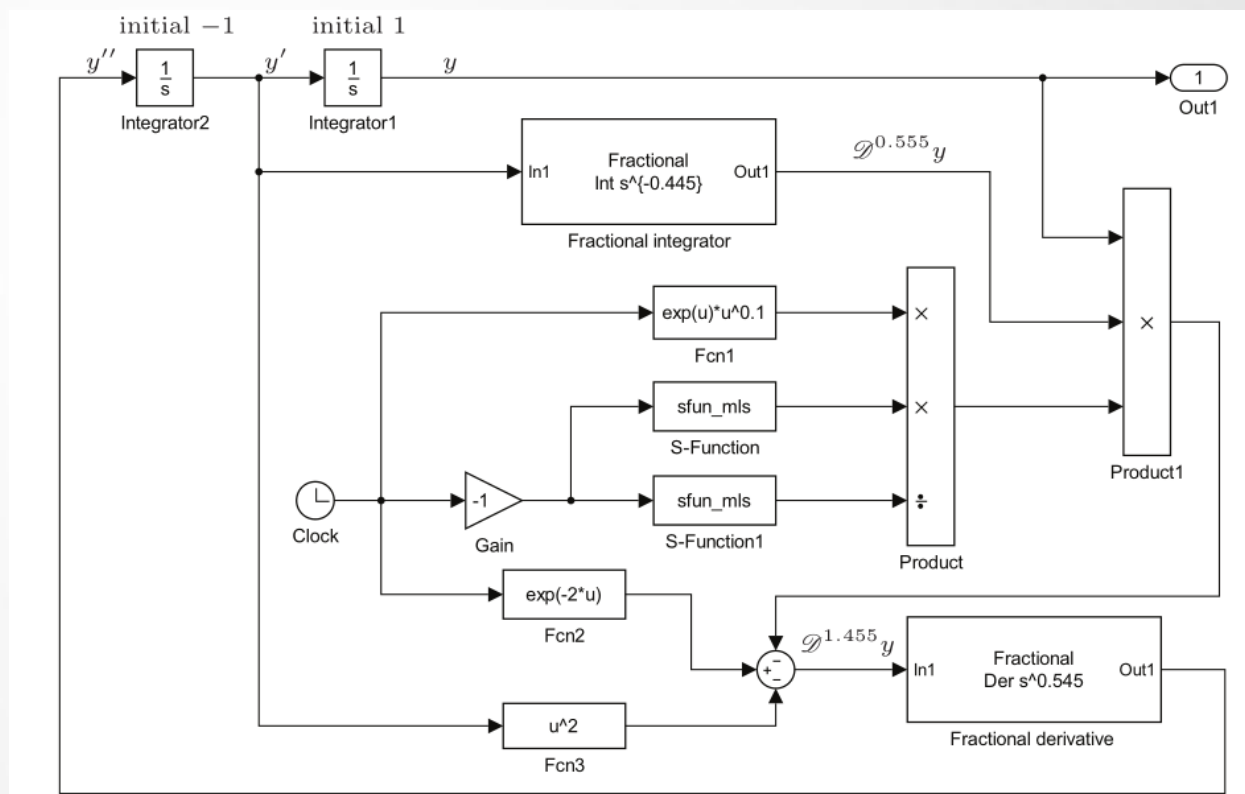
Simulink模型 ${}_0^C \mathcal{D}_t^{1.455} y(t) = -t^{0.1} \frac{\mathcal{E}_{1,1.545}(-t)}{\mathcal{E}_{1,1.445}(-t)} e^t y(t) {}_0^C \mathcal{D}_t^{0.555} y(t) + e^{-2t} - \left[{}_0^C \mathcal{D}_t^1 y(t) \right]^2$

$${}_0^C \mathcal{D}_t^\gamma y(t) = {}_{t_0}^{RL} \mathcal{D}_t^{-(\lceil \gamma \rceil - \gamma)} \left[y^{(\lceil \gamma \rceil)}(t) \right]$$

$${}_0^C \mathcal{D}_t^{0.555} y(t) = {}_{t_0}^{RL} \mathcal{D}_t^{-0.445} [y'(t)]$$

$${}_{t_0}^{RL} \mathcal{D}_t^{\lceil \gamma \rceil - \gamma} \left[{}_0^C \mathcal{D}_t^\gamma y(t) \right] = y^{(\lceil \gamma \rceil)}(t)$$

$${}_{t_0}^{RL} \mathcal{D}_t^{0.545} \left[{}_0^C \mathcal{D}_t^{1.455} y(t) \right] = y''(t)$$





传统微分方程求解算法与Simulink仿真模型求解比较

- 传统方法速度极慢，有时可能需要几小时求解
 - 最好的解 6.8855×10^{-9} , 时间159s
 - Simulink模型求解

(w_b, w_h)	$(10^{-2}, 10^2)$	$(10^{-3}, 10^3)$	$(10^{-4}, 10^4)$	$(10^{-5}, 10^5)$
$p = 5$	5.20×10^{-3}	5.59×10^{-4}	3.58×10^{-5}	9.10×10^{-5}
$p = 6$	5.20×10^{-3}	5.66×10^{-4}	4.71×10^{-5}	2.86×10^{-5}
$p = 7$	5.20×10^{-3}	5.69×10^{-4}	5.59×10^{-5}	1.07×10^{-5}
$p = 8$	5.30×10^{-3}	5.72×10^{-4}	5.76×10^{-5}	5.40×10^{-6}
$p = 9$	5.30×10^{-3}	5.73×10^{-4}	5.76×10^{-5}	5.53×10^{-6}
$p = 10$	5.30×10^{-3}	5.74×10^{-4}	5.76×10^{-5}	6.23×10^{-6}
maximum time	0.156161 s	0.318534 s	2.142945 s	19.480147 s



基准测试问题4 隐式分数阶微分方程

➤ (基准问题4) 隐式分数阶微分方程

$$\begin{aligned} & {}_0^C \mathcal{D}_t^{0.2} y(t) {}_0^C \mathcal{D}_t^{1.8} y(t) + {}_0^C \mathcal{D}_t^{0.3} y(t) {}_0^C \mathcal{D}_t^{1.7} y(t) \\ &= -\frac{1}{8} t \mathcal{E}_{1,1.8}(-t/2) \mathcal{E}_{1,1.2}(-t/2) - \frac{1}{8} t \mathcal{E}_{1,1.7}(-t/2) \mathcal{E}_{1,1.3}(-t/2) \end{aligned}$$

➤ 初始条件 $y(0) = 1, y'(0) = -1/2$

➤ 时间区间 $0 \leq t \leq 10$, 解析解 $y(t) = e^{-t/2}$

➤ 隐式微分方程文献上没有通用的求解方法

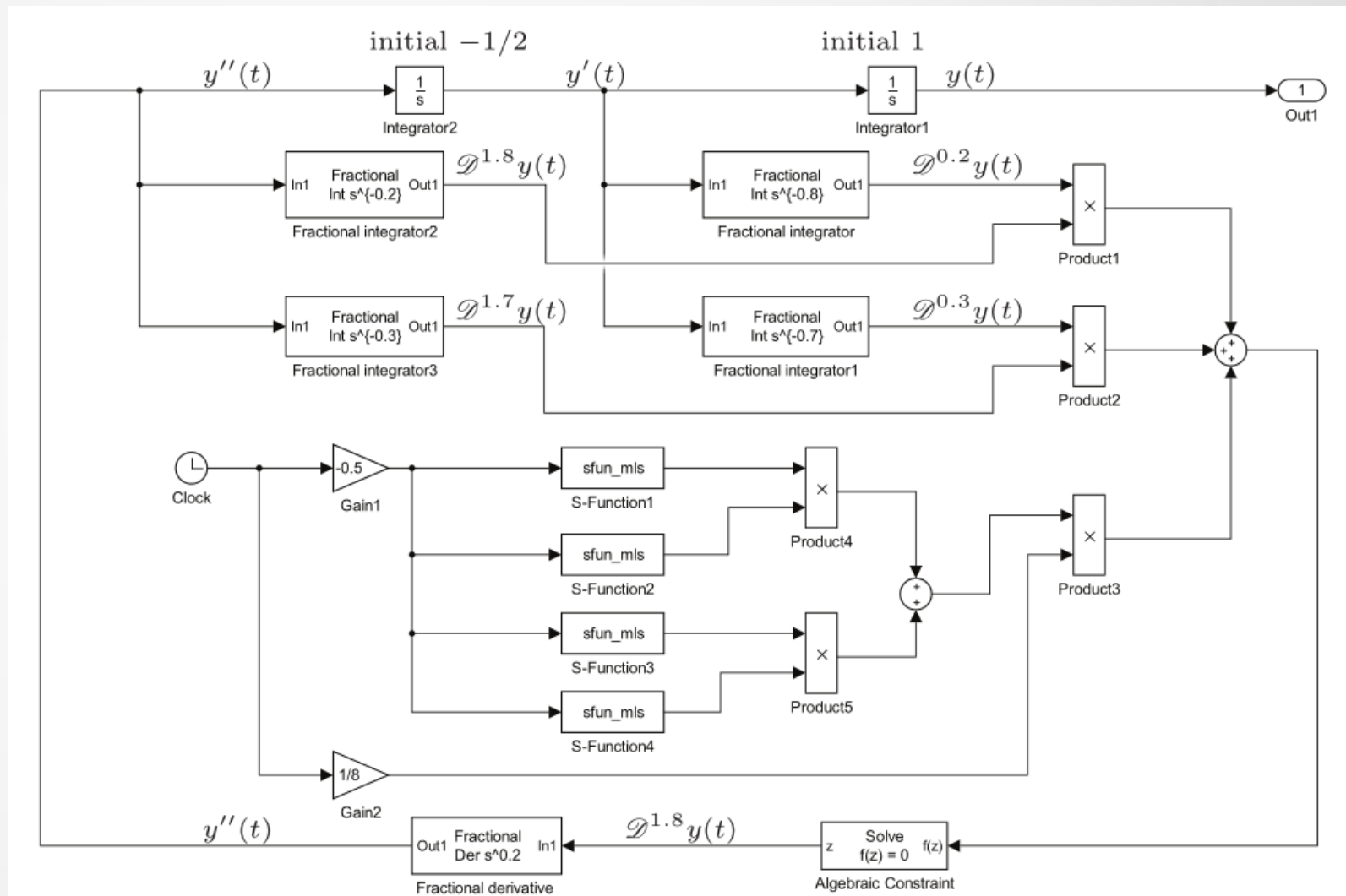


Simulink仿真模型

➤ Simulink模型

➤ 精度 4.11×10^{-5}

➤ 耗时 214s





误差分析

➤ 不同滤波器参数的仿真结果

(w_b, w_h)	$(10^{-2}, 10^2)$	$(10^{-3}, 10^3)$	$(10^{-4}, 10^4)$	$(10^{-5}, 10^5)$
$p = 5$	1.40×10^{-2}	2.30×10^{-3}	7.98×10^{-4}	2.70×10^{-3}
$p = 6$	1.40×10^{-2}	2.30×10^{-3}	4.07×10^{-4}	8.35×10^{-4}
$p = 7$	1.40×10^{-2}	2.30×10^{-3}	3.16×10^{-4}	2.84×10^{-4}
$p = 8$	1.40×10^{-2}	2.30×10^{-3}	2.93×10^{-4}	1.14×10^{-4}
$p = 9$	1.40×10^{-2}	2.30×10^{-3}	2.88×10^{-4}	5.91×10^{-5}
$p = 10$	1.40×10^{-2}	2.30×10^{-3}	2.87×10^{-4}	4.11×10^{-5}
maximum time	0.512608 s	2.597576 s	27.741354 s	214.335601 s

➤ 没有其他方法能求解隐式分数阶微分方程

➤ 速度慢，因为每步仿真需要由 Solve 求解代数方程



分数阶系统的基于**Simulink**的仿真方法

- 定义关键信号与初值
 - 利用整数阶积分器链,并给初值赋值
 - 利用前面的两个定理定义所需的分数阶导数信号
 - 所有初值问题均在整数阶积分器中设置,所以不必再考虑初值
- 构造Simulink仿真模型, 系统的仿真分析
- 仿真结果的验证
 - 一般问题的解析解未知, 所以不能用benchmark验证方法
 - 选择不同的滤波器参数, 如果不同参数下结果一致, 则可以接受
- 理论上可以求解任意复杂的分数阶微分方程

